SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Modal

COURSE NAME: Deep Learning **COURSE CODE: MLA0408**

COURSE OUTCOME COVERED

CO 1: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems.. (BL-6)

Assessment Tool Weightage: 30%

QUESTIONS: **Total Marks:25**

Q1. Transfer Learning

Design a transfer learning framework for a plant disease detection system using a pre-trained CNN model. Justify the choice of layers to freeze and fine-tune, and propose suitable data augmentation techniques to improve classification performance.

Q2. DenseNet and PixelNet

Develop a CNN-based architecture by integrating DenseNet and PixelNet concepts for semantic image segmentation in autonomous driving. Explain how dense connectivity and pixel-wise prediction improve the overall segmentation accuracy.

Q3. Bidirectional RNN and Encoder–Decoder

Create an encoder–decoder sequence-to-sequence model using Bidirectional RNNs for multilingual machine translation. Illustrate the architecture and explain how bidirectional context enhances translation quality.

Q4. BPTT and LSTM

Design an LSTM-based network to predict stock market trends from historical time-series data. Explain how Backpropagation Through Time (BPTT) is applied during training and justify why LSTM is preferred over a standard RNN.

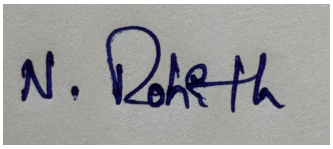
Q5. Integrated Deep Learning Model

Develop a deep learning solution for automatic video caption generation by integrating transfer learning for feature extraction with an LSTM-based encoder–decoder architecture. Justify the role of each component and explain how the complete system generates meaningful captions.

Code of Conduct:

I **NALIPI REDDY ROHITH/192425115**Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Design and Develop Model

Q1. Transfer Learning

Answer

A plant disease detection system can be developed using **Transfer Learning** with a pre-trained CNN such as VGG16, ResNet50, or MobileNet. Transfer learning allows the model to use features learned from a large image dataset and adapt them to plant disease classification.

Proposed Architecture

Input Leaf Image → Pre-trained CNN → Feature Extraction → Fully Connected Layer → Softmax → Disease Class

The pre-trained CNN contains several convolutional layers that learn features such as edges, textures, shapes, and patterns.

Freezing and Fine-Tuning

Initially, the **early convolutional layers are frozen** because they learn general features that are useful for many image classification tasks. The final convolutional layers and newly added classification layers are fine-tuned because they need to learn plant-specific features.

A suitable strategy is:

- Freeze the initial 60–80% of layers.
- Replace the original classification layer.
- Add Global Average Pooling and Dense layers.
- Fine-tune the last few convolutional layers using a small learning rate.

Data Augmentation

To improve generalization, the training images can be augmented using:

- Rotation
- Horizontal and vertical flipping

- Zooming
- Width and height shifting
- Brightness adjustment
- Random cropping

Working

First, plant leaf images are collected and divided into training, validation, and testing sets. The pre-trained CNN extracts useful visual features. The classification layers learn to distinguish healthy leaves from different diseases. Finally, the model predicts the disease class for a new leaf image.

Conclusion

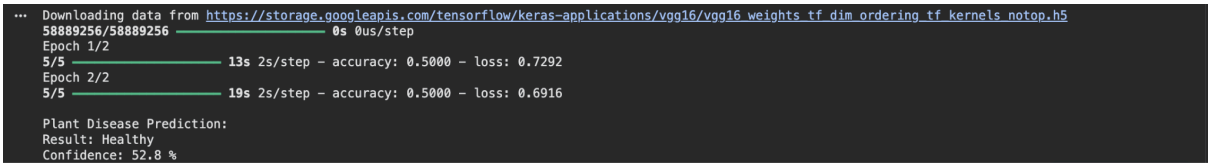
Transfer learning reduces training time and performs well even when the plant disease dataset is relatively small. Freezing general layers and fine-tuning disease-specific layers with data augmentation improves classification accuracy and generalization.

code:

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.applications import VGG16

from tensorflow.keras.layers import Dense, GlobalAveragePooling2D

from tensorflow.keras.models import Model



np.random.seed(42)

tf.random.set_seed(42)



X = np.random.rand(20, 224, 224, 3).astype("float32")

y = np.array([0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1])
```

```
base_model      =      VGG16(weights="imagenet",      include_top=False,
input_shape=(224,224,3))

for layer in base_model.layers:

   layer.trainable = False



x = GlobalAveragePooling2D()(base_model.output)

x = Dense(64, activation="relu")(x)

output = Dense(2, activation="softmax")(x)



model = Model(base_model.input, output)

model.compile(optimizer="adam",  loss="sparse_categorical_crossentropy",
metrics=["accuracy"])

model.fit(X, y, epochs=2, batch_size=4, verbose=1)



test_image = np.random.rand(1,224,224,3).astype("float32")

prediction = model.predict(test_image, verbose=0)

classes = ["Healthy", "Disease"]

result = np.argmax(prediction)



print("\nPlant Disease Prediction:")

print("Result:", classes[result])

print("Confidence:", round(float(prediction[0][result])*100, 2), "%")
```

output:



Q2. DenseNet and PixelNet

Answer

For autonomous driving, semantic segmentation is required to classify every pixel of an image into categories such as road, car, pedestrian, building, and traffic sign. A CNN architecture combining **DenseNet** and **PixelNet concepts** can provide effective feature extraction and pixel-wise prediction.

Proposed Architecture

Input Road Image → DenseNet Feature Extractor → Dense Feature Maps → Pixel-wise Classification → Segmented Image

DenseNet Component

DenseNet uses **dense connectivity**, where each layer receives feature maps from all preceding layers. This improves feature reuse and allows information and gradients to flow efficiently through the network.

Dense connectivity provides:

- Better feature propagation.
- Feature reuse.
- Reduced information loss.
- Improved gradient flow.
- Efficient learning with fewer parameters.

Pixel-wise Prediction

PixelNet focuses on learning representations for individual pixels. For autonomous driving, each pixel must be assigned a semantic class.

For example:

Road → Road class

Car → Car class

Pedestrian → Pedestrian class

Building → Building class

The extracted DenseNet feature maps are converted into pixel-level predictions using suitable convolutional layers.

Working

The input image first passes through DenseNet, which extracts low-level and high-level features. Dense connections preserve information from earlier layers. The resulting feature maps are then used for pixel-wise classification. A final segmentation map is produced in which every pixel receives a class label.

Advantages

- Better feature reuse.
- Improved gradient propagation.
- Accurate boundary detection.
- Effective pixel-level classification.
- Useful for complex road scenes.

Conclusion

Combining DenseNet's dense feature extraction with PixelNet-style pixel-wise prediction provides rich representations and accurate semantic segmentation, making it suitable for autonomous driving applications.

code:

```
import numpy as np

from tensorflow.keras.applications import DenseNet121
```

```
from tensorflow.keras.layers import Conv2D, UpSampling2D

from tensorflow.keras.models import Model



np.random.seed(42)

image = np.random.rand(1,224,224,3).astype("float32")

num_classes = 5



base_model    =    DenseNet121(weights="imagenet",    include_top=False,
input_shape=(224,224,3))

for layer in base_model.layers:

   layer.trainable = False



x          =          Conv2D(128,          (3,3),          padding="same",
activation="relu")(base_model.output)

x = UpSampling2D(size=(8,8))(x)

output       =       Conv2D(num_classes,       (1,1),       padding="same",
activation="softmax")(x)



model = Model(base_model.input, output)

prediction = model.predict(image, verbose=0)

segmentation = np.argmax(prediction[0], axis=-1)



print("Input Image Shape:", image.shape)

print("Segmentation Output Shape:", segmentation.shape)

print("\nPixel-wise segmentation completed.")

print("Sample Pixel Classes:")

print(segmentation[100:105,100:105])
```

output:

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/densenet/densenet121_weights_tf_dim_ordering_tf_kernels_notop.h5
29084464/29084464 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Input Image Shape: (1, 224, 224, 3)
Segmentation Output Shape: (56, 56)

Pixel-wise segmentation completed.
Sample Pixel Classes:
[]
```

Q3. Bidirectional RNN and Encoder–Decoder

Answer

A multilingual machine translation system can be developed using an **Encoder–Decoder architecture with Bidirectional RNNs**. The model converts a sentence from one language into another while considering the context of the entire input sequence.

Architecture

Input Sentence → Embedding → Bidirectional RNN Encoder → Context Representation → RNN/LSTM Decoder → Target Sentence

Encoder

The encoder receives the source sentence word by word. A Bidirectional RNN processes the sequence in two directions:

- Forward direction: beginning → end
- Backward direction: end → beginning

The two hidden representations are combined to obtain a richer representation of each word.

Why Bidirectional RNN?

A normal RNN mainly processes information in one direction. A Bidirectional RNN can use

both previous and future context.

For example, the meaning of a word can depend on words appearing both before and after it. Therefore, bidirectional processing provides more complete contextual information.

Decoder

The decoder receives the encoded representation and generates the translated sentence one word at a time. At each step, the decoder uses the previous generated word and the learned context to predict the next word.

Working

1. Tokenize the source sentence.
2. Convert words into embeddings.
3. Process the sentence using the Bidirectional RNN encoder.
4. Create a context representation.
5. Pass the representation to the decoder.
6. Generate the target sentence sequentially.
7. Train the model by comparing predicted and actual target words.

Advantages

- Uses both past and future context.
- Captures better sentence meaning.
- Improves translation quality.
- Can handle variable-length sequences.
- Suitable for multilingual applications.

Conclusion

A Bidirectional RNN encoder combined with an Encoder–Decoder architecture provides richer contextual information and improves the quality of multilingual machine translation.

code:

```
import numpy as np

import tensorflow as tf
```

```
from tensorflow.keras.models import Model

from tensorflow.keras.layers import Input, Embedding, Bidirectional, LSTM, Dense, RepeatVector


np.random.seed(42)

tf.random.set_seed(42)


vocab_size = 20

sequence_length = 5


X = np.array([[1,2,3,0,0],[1,4,3,0,0],[1,5,6,3,0],[1,2,7,3,0]])

y = np.array([[1,8,9,3,0],[1,10,9,3,0],[1,11,12,9,3],[1,8,13,9,0]])


encoder_input = Input(shape=(sequence_length,))

embedding = Embedding(vocab_size, 32)(encoder_input)

encoder_output = Bidirectional(LSTM(32))(embedding)


context = RepeatVector(sequence_length)(encoder_output)

decoder = LSTM(64, return_sequences=True)(context)

output = Dense(vocab_size, activation="softmax")(decoder)


model = Model(encoder_input, output)

model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])


target = y.reshape(y.shape[0], y.shape[1], 1)

model.fit(X, target, epochs=20, batch_size=2, verbose=0)


test = np.array([[1,2,3,0,0]])

prediction = model.predict(test, verbose=0)

translation = np.argmax(prediction, axis=-1)
```

```
print("Bidirectional Encoder-Decoder Model")

print("Translation:")

print(translation)

print("Prediction Shape:")

print(prediction.shape)
```

output:

```
Bidirectional Encoder-Decoder Model
Translation:
[[9 9 9 0 0]]
Prediction Shape:
(1, 5, 20)
```

Q4. BPTT and LSTM

Answer

An **LSTM-based neural network** can be used to predict stock market trends from historical time-series data. Stock prices depend on information from previous time steps, making LSTM suitable for capturing long-term dependencies.

Proposed Architecture

Historical Stock Data → Preprocessing → LSTM Layers → Dense Layer → Trend Prediction

Input features can include:

- Opening price
- Closing price
- Highest price
- Lowest price
- Trading volume

LSTM

LSTM is a special type of Recurrent Neural Network designed to learn long-term dependencies. It uses a memory cell and three main gates:

1. **Forget Gate** – decides which information should be removed.
2. **Input Gate** – decides which new information should be stored.
3. **Output Gate** – determines the information passed to the next layer.

Backpropagation Through Time (BPTT)

During training, the LSTM is conceptually unfolded across time steps. The prediction error is calculated and propagated backward through these time steps.

The process is:

Forward Pass → Calculate Loss → Propagate Error Back Through Time → Calculate Gradients → Update Weights

BPTT allows the network to learn how previous stock values influence future predictions.

Why LSTM Instead of Standard RNN?

Standard RNNs have difficulty learning long-term dependencies because of vanishing and exploding gradients. LSTM uses gates and a memory cell to preserve important information over long sequences.

Advantages

- Handles sequential data.
- Learns long-term dependencies.
- Reduces vanishing-gradient problems.
- Suitable for stock time-series prediction.

Conclusion

An LSTM trained using BPTT can effectively learn temporal patterns in historical stock data. LSTM is preferred over standard RNN because its memory mechanism allows it to retain important long-term information.

code:

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import LSTM, Dense



np.random.seed(42)

tf.random.set_seed(42)



prices                                                                  =
np.array([100,102,104,103,106,108,110,109,112,115,117,116,120,122,125,1
24,128,130,133,135], dtype=float)



minimum = prices.min()

maximum = prices.max()

scaled = (prices - minimum) / (maximum - minimum)



X = []

y = []

sequence_length = 3



for i in range(len(scaled)-sequence_length):

   X.append(scaled[i:i+sequence_length])

   y.append(scaled[i+sequence_length])



X = np.array(X)

y = np.array(y)
```

```
X = X.reshape(X.shape[0], X.shape[1], 1)



model = Sequential([LSTM(32, input_shape=(3,1)), Dense(1)])

model.compile(optimizer="adam", loss="mse")

model.fit(X, y, epochs=30, batch_size=4, verbose=0)



print("LSTM Training Completed.")



test_input = scaled[-3:].reshape(1,3,1)

prediction = model.predict(test_input, verbose=0)

predicted_price = prediction[0][0]*(maximum-minimum)+minimum



print("Predicted Next Closing Price:", round(float(predicted_price),
2))
```

output:

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_
  super().__init__(**kwargs)
LSTM Training Completed.
Predicted Next Closing Price: 132.43
```

Q5. Integrated Deep Learning Model

Answer

Automatic video caption generation combines **Transfer Learning** for visual feature extraction with an **LSTM Encoder–Decoder** for generating captions. The system analyzes video frames and produces a meaningful textual description.

Proposed Architecture

Video → Frame Extraction → Pre-trained CNN → Visual Features → LSTM Encoder → LSTM Decoder → Caption

Step 1: Video Processing

The video is divided into a sequence of frames. Important frames are selected at suitable time intervals to reduce computational cost.

Step 2: Transfer Learning

A pre-trained CNN such as VGG16, ResNet or Inception is used to extract visual features from each frame. The CNN has already learned useful visual patterns from a large image dataset.

The CNN converts each frame into a compact feature vector.

Step 3: LSTM Encoder

The sequence of visual feature vectors is provided to the LSTM encoder. The LSTM learns the temporal relationship between different frames.

For example, a video may contain:

Person → picking up ball → throwing ball

The LSTM learns the order of these events.

Step 4: LSTM Decoder

The decoder generates the caption word by word. It starts with a start token and predicts the next word based on the learned video representation and previously generated words.

Example:

"A person is throwing a ball."

Role of Each Component

Transfer Learning: Extracts powerful visual features without training a CNN from scratch.

LSTM Encoder: Understands temporal relationships between video frames.

LSTM Decoder: Converts the learned representation into a natural-language caption.

Advantages

- Reduces CNN training requirements.
- Captures both spatial and temporal information.
- Generates meaningful captions.
- Can be applied to surveillance, sports and accessibility systems.

Conclusion

The combination of transfer learning and LSTM Encoder–Decoder provides an effective solution for video caption generation by understanding visual content, learning temporal relationships and generating meaningful natural-language descriptions.

code:

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.applications import MobileNetV2

from tensorflow.keras.layers import Input, LSTM, Dense

from tensorflow.keras.models import Model



np.random.seed(42)

tf.random.set_seed(42)



cnn = MobileNetV2(weights="imagenet", include_top=False, pooling="avg",
input_shape=(224,224,3))

for layer in cnn.layers:

   layer.trainable = False
```

```
frames = np.random.rand(5,224,224,3).astype("float32")

features = cnn.predict(frames, verbose=0).reshape(1,5,1280)


encoder_input = Input(shape=(5,1280))

encoder = LSTM(128, return_state=True)

encoder_output, state_h, state_c = encoder(encoder_input)


decoder_input = Input(shape=(5,9))

decoder_output = LSTM(128, return_sequences=True)(

   decoder_input, initial_state=[state_h, state_c]

)

output = Dense(9, activation="softmax")(decoder_output)


model = Model([encoder_input, decoder_input], output)

model.compile(optimizer="adam", loss="categorical_crossentropy")


decoder_data = np.random.rand(1,5,9).astype("float32")

prediction = model.predict([features, decoder_data], verbose=0)


caption_words                                                    =
["a","boy","is","playing","football","girl","running","man","walking"]

predicted_words = np.argmax(prediction[0], axis=1)

caption = [caption_words[i] for i in predicted_words]


print("Video Caption Model Created Successfully.")

print("Feature Shape:", features.shape)
```

```
print("Prediction Shape:", prediction.shape)

print("Generated Caption:")

print(" ".join(caption))
```

output:

```
...  Video Caption Model Created Successfully.
     Feature Shape: (1, 5, 1280)
     Prediction Shape: (1, 5, 9)
     Generated Caption:
     walking walking walking walking walking
```